CSC265 Notes

Max Xu

'25 Fall

Contents

1 Tutorial 1 (Sept 12, 2025) 2

2 Tutorial 2 (Sep 19, 2025) 3

3 Tutorial 3 (Sept 26, 2025) 4

4 Tutorial 4 (Oct 03, 2025) 5

5 Tutorial 5 (Oct 10, 2025) 6

6 Tutorial 6 (Nov 07, 2025) 9

§1 Tutorial 1 (Sept 12, 2025)

Problem 1.1. Show that building a binomial heap from an array A of n elements takes $O(n)$ time.

Problem 1.2. Suppose you want to implement a dictionary using a direct access table in some universe $U = \{1, \cdots, n\}$ of keys. Unfortunately every array that you initialize will likely contain garbage information. Describe an implementation that takes $O(1)$ time to initialize and support $O(1)$ time search, insert, and delete.

Problem 1.3.

Question 3

```
FUNC(x)
result = 1
while x > 0
    A = 2
    y = 1
    while 2y < x
        y = 2y
        A = A^2
    x = x - y
    result = result · A
return result
```

(a) Suppose that x is an integer and $x \geq 1$. Let $k = \lceil \log_2(x+1) \rceil$ be the number of bits needed to write x in binary. Give a tight asymptotic upper bound in terms of k on the worst-case running time of FUNC(x). You can assume that any arithmetic operation takes constant time. Briefly justify your answer.

Problem 1.4. Given an implementation for the predecessor operation on a binary search tree T, which gives a key k, return a pointer to a node T with the largest key less than k.

§2 Tutorial 2 (Sep 19, 2025)

Theorem 2.1. Let f be a field that augments an AVL tree T of n nodes, and suppose that the value of f for each node x depends on only the information in nodes x, $x.left$, $x.right$. Then we can maintain the values of f in all nodes of T during insertion and deletion without asymptotically affecting the $O(\log n)$ performance of these operations.

Examples of such fields f include

size $x.size = x.right.size + x.left.size + 1$

max the maximum data in the subtree rooted at x. $x.max = \max\{x.left, x.right, x.data\}$, or if we are max-ing over the keys, $x.max = x.right.max$ if $x.right \neq \textsc{Nil}$, $x.key$ otherwise

sum sum of all keys in the subtree

min minimum data/key in the subtree

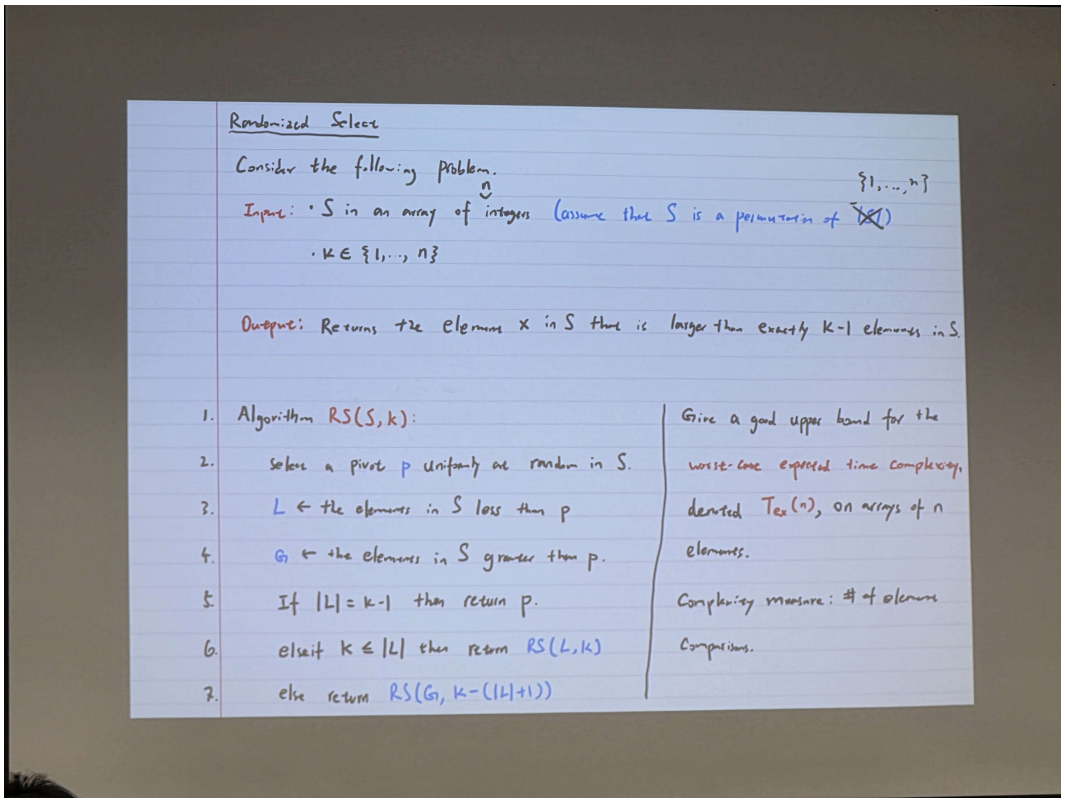
Problem 2.2. Count the number of inversions of array A of length n. An inversion is a pair (i, j) with $i < j$ and $A[i] > A[j]$.

Problem 2.3. Given an example of an n-node AVL tree such that it contains a node, which when deleted from the tree requires $\Omega(\log_2 n)$ rotations to rebalance the tree.

Hint: give a recursive definition for a family of trees $\{T_k\}_{k\in\mathbb{N}}$ of minimal size AVL tree of height h, for $h \in \mathbb{N}$, and consider deletion from T_h.

§3 Tutorial 3 (Sept 26, 2025)

Problem 3.1.



Randomized Select

Consider the following problem.

Input: · S is an array of integers (assume that S is a permutation of $\{1, \ldots, n\}$)

· $k \in \{1, \ldots, n\}$

Output: Returns the element x in S that is larger than exactly $k-1$ elements in S.

```
Algorithm RS(S, k):
    Select a pivot p uniformly at random in S.
    L ← the elements in S less than p
    G ← the elements in S greater than p.
    If |L| = k-1 then return p.
    elseif k ≤ |L| then return RS(L, k)
    else return RS(G, k-(|L|+1))
```

Give a good upper bound for the worst-case expected time complexity, denoted $T_{ex}(n)$, on arrays of n elements.

Complexity measure: # of element comparisons.

We want to count the number of comparisons. We must first define a random variable, let us define $X_{i,j,k}(\alpha)$ be the indicator variable for if a comparison between element i and j occurred when α is the sequence of random pivots chosen by the algorithm, with k being the pivot picked. Then

$$t_n(\alpha) = \sum_{1 \le i < j \le n} X_{i,j,k}(\alpha)$$

thus

$$T_{EX}(n) = \mathop{\mathbb{E}}_{\alpha}[t_n(\alpha)] = \sum_{1 \le i < j \le n} \mathop{\mathbb{E}}_{\alpha}[t_n(\alpha)] = \sum_{1 \le i < j \le n} \Pr[X_{i,j,k} = 1]$$

We partition into cases as follows:

Case $i < j < k$: We realize that any contiguous part of $\{i, \cdots, k\}$ 'remains together' through successive recursive calls, until the first time a pivot is chosen among them.

Let p be the new pivot that is chosen. If $p \neq i$ and $p \neq j$, then i and j are not compared, since j would be in G, and i would be in L. However, if $p = i$ or $p = j$, then p is compared against the rest of S, so the comparison between i and j must occur.

$$\Pr[X_{i,j,k} = 1] = \Pr[p = i \text{ or } p = j] = 2 \cdot \frac{1}{k - i + 1}$$

Case $k < i < j$

This case is similar, giving us $\Pr[X_{i,j,k} = 1] = 2 \cdot \frac{1}{k-i+1}$

§4 Tutorial 4 (Oct 03, 2025)

Problem 4.1.

1. A family $\mathcal{H}$ of hash functions from a universe U to $\{0, \ldots, m-1\}$ is 3-universal if, for all distinct $x_1, x_2, x_3 \in U$ and all $y_1, y_2, y_3 \in \{0, \ldots, m-1\}$,

strongly k-wise independent hash family

$$\text{Prob}_{h \in \mathcal{H}}[h(x_1) = y_1, h(x_2) = y_2, \text{ and } h(x_3) = y_3] = \frac{1}{m^3}.$$

(a) Prove that any 3-universal family of hash functions is universal.

(b) Prove that if $U = \{0, \ldots, m-1\}$ and m is prime, then the family $\mathcal{H} = \{h_{a,b,c} \mid a, b, c \in U\}$ is 3-universal, where $h_{a,b,c}(x) = (ax^2 + bx + c) \bmod m$.

Problem 4.2.

2. One application of universal hashing is called "fingerprinting." Let's say one person wants to send a file to another person over a network and wants to check that the file wasn't corrupted during the transmission. Both the sender and the receiver have access to a universal class of hash functions $\mathcal{H} = \{h_1, h_2, \ldots, h_k\}$. The two people proceed as follows:

- The sender sends the file f to the receiver.
- The receiver receives a file f' which may or may not be the same as f depending on whether it was corrupted during transmission. The receiver chooses a random i from $\{1, \ldots, k\}$ and computes $h_i(f')$. The receiver sends i and $h_i(f')$ to the sender.
- The sender computes $h_i(f)$ and checks to see if it matches $h_i(f')$.

(a) Assume the file is K bits long and that each h in $\mathcal{H}$ maps $\{0,1\}^K$ to $\{0, 1, \ldots, m-1\}$. How many bits are sent across the network during this whole transaction? Note the exact number of bits, do not use asymptotic notation here.

(b) Assume that f is corrupted during transmission so that $f' \neq f$. What is the probability that the fingerprinting produces a "false positive": that is, the probability that $h_i(f) = h_i(f')$?

(c) Describe a way to significantly reduce the probability from part (b). How many bits are transmitted now?

§5 Tutorial 5 (Oct 10, 2025)

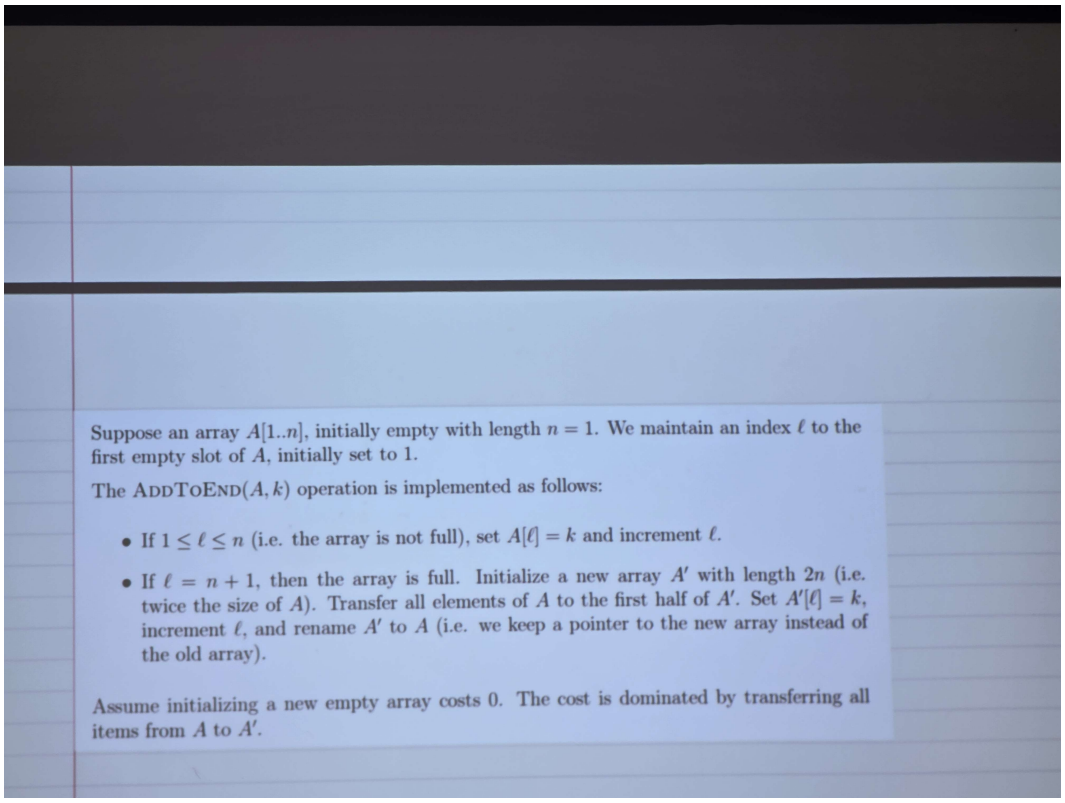
Formally, let σ be any sequence of m operations. Have ϵ_i be the amortized cost assigned to the i-th operation, and c_i be the actual cost of the i-th operation. Then

$$\sum_{i=1}^{n} \epsilon_i \geq \sum_{i=1}^{n} c_i + (\text{credits stored in the data structure})$$

ϵ_i is allowed to be wasteful, because sometimes it's simpler to 'count less money'. We introduce a credit invariant, in an effort to attach a chunk of the whole data structure's balance to say, a single node, which simplifies computations.

Definition 5.1 (Credit Invariant)**.** A rule that says how many credits is stored in parts of the data structure.

Problem 5.2.



Suppose an array $A[1..n]$, initially empty with length $n = 1$. We maintain an index ℓ to the first empty slot of A, initially set to 1.

The AddToEnd(A, k) operation is implemented as follows:

- If $1 \leq \ell \leq n$ (i.e. the array is not full), set $A[\ell] = k$ and increment ℓ.
- If $\ell = n + 1$, then the array is full. Initialize a new array A' with length $2n$ (i.e. twice the size of A). Transfer all elements of A to the first half of A'. Set $A'[\ell] = k$, increment ℓ, and rename A' to A (i.e. we keep a pointer to the new array instead of the old array).

Assume initializing a new empty array costs 0. The cost is dominated by transferring all items from A to A'.

Our goal is to show that AddToEnd has amortized cost $O(1)$, meaning on average, it takes a constant amount of work to append to this data structure. We perform analysis as follows:

Accounting Method

Step 1 Define what \$ pays for (i.e. what is a step)
Here, we have 1 \$ pay for writing an in integer into an array.

Step 2 Determine what we charge for each operation
We charge \$2 for each call to AddToEnd called.

Step 3 Define a credit invariant
Credit Invariant: Each nonempty entry in the second half of the array A has \$3 allocated to it.

Step 4 Prove the credit invariant by induction

Want to show that our credit invariant holds, that each non-empty entry in the second half of A has \$2, as well as this inequality, where ϵ_i is the amortized cost, and c_i is the actual cost.

$$\sum_{i=1}^{m} \epsilon_i \geq \sum_{i=1}^{m} c_i + (\text{sum of credits in the data structure})$$

Base Case: $n = 1$, with each $A[1] = \textsc{Nil}$.
Since no operation was performed, $m = 0$. Since $A[1] = \textsc{Nil}$, the data structure have 0 credits. Thus both the invariant and the inequality holds.

Induction Step: Assume the invariant and inequality both hold for some m. Want to show they still hold for $m + 1$. We proceed by cases:

Case 1: Suppose A is not full (i.e. $\ell = m \leq n$ at the start of the $(m + 1)$-th operation). Assume that \$3 is charged for the $(m + 1)$-th AddToEnd, and use \$1 to pay for writing k to $A[\ell]$. Allocate the remaining \$2 of credit to $A[\ell]$.

Since the credit invariant was maintained on all other elements in the second half of A, and $A[\ell]$ is the only empty entry that becomes non-empty during this operation, we conclude that the credit invariant holds after the $(m + 1)$-th operation. We also have:

$$\begin{aligned}
\sum_{i=1}^{m+1} \epsilon_i &= \sum_{i=1}^{m} \epsilon_i + 3 \\
&\geq \sum_{i=1}^{m} c_i + (\text{old sum of credits}) + 3 \\
&= \sum_{i=1}^{m} c_i + (\text{new sum of credits} - 2) + 3 \\
&= \sum_{i=1}^{m+1} c_i
\end{aligned}$$

Case 2: Suppose that A is full (i.e. $\ell = m = n + 1$ at the start of the $(m + 1)$-th operation). Copy the elements of A into the first half of A', which costs $\$n$. We pay for the transfer using the $\frac{n}{2} \cdot \$2$ of credits in the second half of A. This amount of credits exists by the invariant, since we know that A is full.

Among the \$3 collected from calling the $(m + 1)$-th call of AddToEnd, use \$1 to write k into $A'[\ell]$, and store \$2 to $A'[\ell]$. As $A'[\ell]$ is the only such entry in the second half of A', and has \$2 allocated to it, the credit invariant holds. Returning to the inequality,

$$
\begin{aligned}
\sum_{i=1}^{m+1} \epsilon_i &= \sum_{i=1}^{m} \epsilon_i + 3 \\
&\geq \sum_{i=1}^{m} c_i + \underbrace{(\text{old credits in DS})}_{\geq n} + 3 \\
&= \sum_{i=1}^{m} c_i + (\text{new credits in DS}) + n - 2 + 3 \\
&= \sum_{i=1}^{m} c_i + n + 1 + (\text{new credits in DS}) \\
&= \sum_{i=1}^{m+1} c_i + (\text{new credits in DS})
\end{aligned}
$$

Step 5 State the conclusion.

By the previous inequality, the sum of the amortized costs (ϵ_i) is at least the sum of the actual costs (c_i) of the m operations. Since each AddToEnd operation has amortized cost \$3, the (actual) cost for performing a sequence of m operations is upper bounded by $3m$.

§6 Tutorial 6 (Nov 07, 2025)

The graph is connected.

Proof. We prove both directions.

($\Rightarrow$) Suppose that G is bipartite. Then there exists partition of V, being X, Y, such that there is no edge between vertices in the same partition. For any cycle $v_1, \cdots, v_n \in G$, adjacent vertices in this sequence must be in different partitions. Thus odd number indices are in 1 partition, and even number indices are in the other. Since we have the edge $\{v_1, v_n\}$, so v_n and v_1 must be in different partitions. Hence n is even.

($\Leftarrow$) Suppose G does have odd length cycles, and G is bipartite. Pick some $s \in V$ and for all $v \in V$, define $\delta(s, v)$ to be the shortest distance between s and v in G.

- Take $X = \{v \in V : \delta(s, v) \text{ is even}\}$.
- $Y = \{v \in V : \delta(s, v) \text{ is odd}\}$.
  Since G is connected, $X \cup Y = V$.
- Let P_u be a shortest path from u to S.
- Let P_v be a shortest path from v to S.
- w be the first common vertex on P_u and P_v.

Consider the path $u \to \cdots \to w \to \cdots \to v$. We can obtain a cycle, since $\{u, v\} \in E$. Have:

$$\delta(s, u) - \delta(s, w) + \delta(s, v) - \delta(s, w) + 1$$

Which is odd, and a contradiction. TODO: Why is $u + v$ even???

$\square$